

高级算法复习讲解

目录

01 复杂度与最优性	5
题目原文	5
考查类型	5
核心概念	5
典型例子	5
答题模板	5
例题	6
常见误区	6
02 渐进符号定义与证明	7
题目原文	7
考查类型	7
三个定义	7
证明方法	7
示例	7
常用性质	8
例题	8
常见误区	9
03 二阶线性递归关系求解	10
题目原文	10
考查类型	10
解题思路	10
两个不同特征根	10
重根	10
斐波拉契数列例子	11
渐进复杂度判断	11
答题模板	11

例题	12
常见误区	13
04 平滑函数与主定理	14
题目原文	14
考查类型	14
递推含义	14
主定理三种情形	14
平滑函数的作用	15
例子	15
答题模板	16
例题	16
常见误区	18
05 堆的定义、数组表示与操作	19
题目原文	19
考查类型	19
堆的定义	19
数组表示	19
主要操作	20
典型伪代码	20
常见应用	21
例题	21
常见误区	22
06 快排平均复杂度递推分析	23
题目原文	23
考查类型	23
递推来源	23
解法：乘 n 再相减	23
答题模板	24
例题	25
常见误区	26
07 对抗策略分析	27
题目原文	27
考查类型	27
对抗策略含义	27

同时求最大值和最小值	27
下界证明思路	28
求第二大元素	28
第二大元素的下界思路	28
例题	29
常见误区	30
08 线性时间选择算法	31
题目原文	31
考查类型	31
问题定义	31
随机选择算法思想	31
确定性线性时间选择算法	31
为什么是线性时间	32
应用	32
伪代码框架	32
例题	33
常见误区	34
09 平摊分析	35
题目原文	35
考查类型	35
平摊分析含义	35
三种常见方法	35
MultiPop Stack	35
DeleteLargerHalf	36
两个栈模拟队列	36
例题	37
常见误区	39
10 DFS 与 BFS 图搜索	40
题目原文	40
考查类型	40
DFS 思想	40
BFS 思想	40
topo 序	41
任务调度	41

关键路径	41
强连通区域问题	42
例题	42
常见误区	43
11 贪心法	44
题目原文	44
考查类型	44
贪心法思想	44
证明思路	44
换硬币问题	44
单源最短路径	45
Dijkstra 正确性证明	45
例题	46
常见误区	47
12 动态规划	48
题目原文	48
考查类型	48
动态规划思想	48
最长公共子序列	48
抽牌拿取最大值问题	49
回文判断	50
回文分割	50
例题	50
常见误区	52

01 复杂度与最优性

题目原文

算法的最坏复杂度/平均复杂度/最优性，概念和举例

考查类型

概念题，也会要求配例子说明。

核心概念

最坏复杂度描述规模为 n 的所有输入中，算法运行代价的最大值：

$$W(n) = \max\{ T(I) \mid I \text{ 是规模为 } n \text{ 的输入} \}$$

它回答的是：在最不利输入下，算法最多需要多少时间或空间。最坏复杂度常用于给出算法的稳定保证。

平均复杂度描述规模为 n 的输入在某个概率分布下的期望代价：

$$A(n) = E[T(I)] = \sum P(I)T(I)$$

平均复杂度必须说明输入分布。没有输入分布时，不能直接写平均复杂度的精确定义。

最优性是算法和问题下界之间的关系。如果某个问题在指定计算模型下有下界 $L(n)$ ，而某算法的复杂度达到 $O(L(n))$ ，则该算法在该模型下是渐进最优的。

典型例子

比较排序模型中，任何基于比较的排序算法都需要 $\Omega(n \log n)$ 次比较。归并排序和堆排序的最坏时间复杂度是 $O(n \log n)$ ，因此它们在比较排序模型下是渐进最优的。

快速排序的最坏时间复杂度是 $O(n^2)$ ，平均时间复杂度是 $O(n \log n)$ 。这说明同一个算法在不同分析口径下会得到不同结论。

答题模板

1. 先写定义：最坏是最大代价，平均是期望代价，最优是达到问题下界。
2. 再写公式： $\max$ 、 E 、上下界匹配。
3. 最后举例：比较排序、快速排序、查找最小值等。

例题

例题 1

题目：算法 A 的最坏时间复杂度为 $\Theta(n^2)$ ，平均时间复杂度为 $\Theta(n \log n)$ ；算法 B 的最坏时间复杂度为 $\Theta(n \log n)$ ，平均时间复杂度为 $\Theta(n \log^2 n)$ 。分别从最坏保证和平均表现角度比较二者。

解答：

从最坏保证看，算法 B 更好，因为：

$$n \log n = o(n^2)$$

从平均表现看，算法 A 更好，因为：

$$n \log n = o(n \log^2 n)$$

这说明“更好”必须说明分析口径。最坏复杂度强调上界保证，平均复杂度强调给定输入分布下的期望表现。

例题 2

题目：说明归并排序在比较排序模型下的最优性。

解答：

比较排序模型中，任何排序算法都需要：

$$\Omega(n \log n)$$

次比较。归并排序的最坏比较次数为：

$$O(n \log n)$$

上下界匹配，因此归并排序在比较排序模型下是渐进最优的。

常见误区

不要把平均复杂度写成“常见输入下的复杂度”。平均复杂度是数学期望，必须依赖输入分布。

不要只说某算法是最优的。必须说明在哪个问题、哪个计算模型、哪个下界意义下最优。

02 渐进符号定义与证明

题目原文

O 、 Ω 、 Θ 的定义和相关函数证明

考查类型

概念题和证明题。

三个定义

设 $f(n)$ 和 $g(n)$ 是非负函数。

$f(n) = O(g(n))$ 表示 f 被 g 渐进上界控制：

存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ ，有 $0 \leq f(n) \leq c g(n)$

$f(n) = \Omega(g(n))$ 表示 f 被 g 渐进下界控制：

存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ ，有 $0 \leq c g(n) \leq f(n)$

$f(n) = \Theta(g(n))$ 表示二者同阶：

存在正常数 c_1, c_2 和 n_0 ，使得对所有 $n \geq n_0$ ，有 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

等价地：

$f(n) = \Theta(g(n))$ 当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$

证明方法

证明 O ：找到一个常数 c ，让 $f(n) \leq cg(n)$ 在足够大的 n 上成立。

证明 Ω ：找到一个常数 c ，让 $f(n) \geq cg(n)$ 在足够大的 n 上成立。

证明 Θ ：同时证明上界和下界。

示例

证明 $3n^2 + 5n + 7 = O(n^2)$ 。

当 $n \geq 1$ 时：

$$3n^2 + 5n + 7 \leq 3n^2 + 5n^2 + 7n^2 = 15n^2$$

取 $c = 15, n_0 = 1$ ，定义满足，因此结论成立。

证明 $3n^2 + 5n + 7 = \Omega(n^2)$ 。

当 $n \geq 1$ 时：

$$3n^2 + 5n + 7 \geq 3n^2$$

取 $c = 3, n_0 = 1$ ，定义满足，因此结论成立。

由上界和下界同时成立，得到：

$$3n^2 + 5n + 7 = \Theta(n^2)$$

常用性质

传递性：

$$f = O(g), g = O(h) \Rightarrow f = O(h)$$

同阶对称性：

$$f = \Theta(g) \Rightarrow g = \Theta(f)$$

多项式主项支配：

$$a_d n^d + \dots + a_1 n + a_0 = \Theta(n^d)$$

这里要求最高次项系数 $a_d > 0$ 。

例题

例题 1

题目：用定义证明：

$$5n^3 + 2n^2 + 7 = O(n^3)$$

解答：

当 $n \geq 1$ 时：

$$5n^3 + 2n^2 + 7 \leq 5n^3 + 2n^3 + 7n^3 = 14n^3$$

取：

$$c = 14$$

$$n_0 = 1$$

由 O 的定义，结论成立。

例题 2

题目：用定义证明：

$$5n^3 + 2n^2 + 7 = \Theta(n^3)$$

解答：

上界由例题 1 得到：

$$5n^3 + 2n^2 + 7 \leq 14n^3$$

下界为：

$$5n^3 + 2n^2 + 7 \geq 5n^3$$

取：

$$c_1 = 5$$

$$c_2 = 14$$

$$n_0 = 1$$

满足：

$$c_1 n^3 \leq 5n^3 + 2n^2 + 7 \leq c_2 n^3$$

所以：

$$5n^3 + 2n^2 + 7 = \Theta(n^3)$$

常见误区

不要只代入几个数值说明渐进关系。渐进证明必须给出常数和阈值。

不要把 O 当成等号意义上的相等。 O 表示上界关系，不表示精确增长。

03 二阶线性递归关系求解

题目原文

递归关系求解：

$$a_n = r_1 a_{n-1} + r_2 a_{n-2}$$

形似斐波拉契数列。

考查类型

证明和计算题。

解题思路

这是二阶线性齐次递推。标准方法是写出特征方程：

$$x^2 = r_1 x + r_2$$

整理为：

$$x^2 - r_1 x - r_2 = 0$$

求出特征根后，再根据根的情况写通解。

两个不同特征根

若特征根为 α 和 β ，且 $\alpha \neq \beta$ ，则通解为：

$$a_n = A\alpha^n + B\beta^n$$

其中 A 和 B 由初始条件决定，例如 a_0 和 a_1 。

重根

若特征方程只有一个重根 α ，则通解为：

$$a_n = (A + Bn)\alpha^n$$

同样用初始条件求出 A 和 B。

斐波拉契数列例子

斐波拉契数列满足：

$$F_n = F_{n-1} + F_{n-2}$$

对应 $r_1 = 1, r_2 = 1$, 特征方程为：

$$x^2 - x - 1 = 0$$

两个根为：

$$\varphi = (1 + \sqrt{5})/2$$

$$\psi = (1 - \sqrt{5})/2$$

所以：

$$F_n = A\varphi^n + B\psi^n$$

再由初始条件确定 A 和 B。

渐进复杂度判断

如果两个根的绝对值不相等，且主导根对应系数不为 0，则 a_n 的增长由绝对值最大的根控制。

例如斐波拉契数列中：

$$|\varphi| > |\psi|$$

因此：

$$F_n = \Theta(\varphi^n)$$

答题模板

1. 写特征方程 $x^2 - r_1x - r_2 = 0$ 。
2. 求特征根。
3. 按不同根或重根写通解。
4. 代入初始条件求常数。
5. 如题目要求渐进阶，比较特征根绝对值。

例题

例题 1

题目：求解递推：

$$a_n = 3a_{n-1} - 2a_{n-2}$$

$$a_0 = 1$$

$$a_1 = 3$$

解答：

特征方程为：

$$x^2 - 3x + 2 = 0$$

分解得：

$$(x - 1)(x - 2) = 0$$

两个不同特征根为 1 和 2，所以：

$$a_n = A \cdot 1^n + B \cdot 2^n = A + B2^n$$

代入初始条件：

$$A + B = 1$$

$$A + 2B = 3$$

解得：

$$A = -1$$

$$B = 2$$

因此：

$$a_n = 2^{n+1} - 1$$

例题 2

题目：求解递推：

$$a_n = 2a_{n-1} - a_{n-2}$$

$$a_0 = 1$$

$$a_1 = 2$$

解答：

特征方程为：

$$x^2 - 2x + 1 = 0$$

即：

$$(x - 1)^2 = 0$$

特征根 1 是重根，通解为：

$$a_n = (A + Bn)1^n = A + Bn$$

代入初始条件：

$$A = 1$$

$$A + B = 2$$

解得：

$$A = 1$$

$$B = 1$$

因此：

$$a_n = n + 1$$

常见误区

不要把所有二阶递推都直接写成斐波拉契形式。只有 $r_1 = 1, r_2 = 1$ 且初始条件对应时，才是标准斐波拉契数列。

不要漏掉重根情形。重根时必须出现 $n\alpha^n$ 项。

04 平滑函数与主定理

题目原文

了解平滑函数性质，应用主定理求解一般式：

$$T(n) = bT(n/c) + f(n)$$

考查类型

概念题和递推分析题。

递推含义

递推式：

$$T(n) = bT(n/c) + f(n)$$

表示一个规模为 n 的问题被分成 b 个规模为 n/c 的子问题，合并或额外处理代价为 $f(n)$ 。

其中通常有：

$$b \geq 1$$

$$c > 1$$

关键比较对象是：

$$n^{\log_c b}$$

它表示递归树中叶子层的总规模量级。

主定理三种情形

令：

$$\alpha = \log_c b$$

第一种： $f(n)$ 比 n^α 多项式意义下更小。

$$f(n) = O(n^{\alpha-\varepsilon})$$

其中 $\varepsilon > 0$ ，则：

$$T(n) = \Theta(n^\alpha)$$

第二种： $f(n)$ 与 n^α 同阶。

$$f(n) = \Theta(n^\alpha)$$

则：

$$T(n) = \Theta(n^\alpha \log n)$$

第三种： $f(n)$ 比 n^α 多项式意义下更大，并满足正则条件。

$$f(n) = \Omega(n^{\alpha+\varepsilon})$$

且存在常数 $q < 1$ ，使得：

$$bf(n/c) \leq qf(n)$$

则：

$$T(n) = \Theta(f(n))$$

平滑函数的作用

在主定理相关证明中，平滑性质用于处理 n 不是 c 的整数幂时的情况，并保证函数在按常数比例缩放时增长不发生剧烈跳变。

常见的平滑函数包括：

$$n^d$$

$$n^d \log^k n$$

其中 d 和 k 是常数，且函数在讨论区间内非负。

这些函数满足按常数倍缩放时仍保持同一量级：

$$f(cn) = \Theta(f(n))$$

例子

归并排序：

$$T(n) = 2T(n/2) + n$$

这里：

$$b = 2$$

$$c = 2$$

$$\alpha = \log_2 2 = 1$$

$$f(n) = n = \Theta(n^1)$$

属于第二种情形：

$$T(n) = \Theta(n \log n)$$

二分查找：

$$T(n) = T(n/2) + 1$$

这里：

$$b = 1$$

$$c = 2$$

$$\alpha = \log_2 1 = 0$$

$$f(n) = 1 = \Theta(n^0)$$

属于第二种情形：

$$T(n) = \Theta(\log n)$$

答题模板

1. 写出 b 、 c 、 $f(n)$ 。
2. 计算 $\alpha = \log_c b$ 。
3. 比较 $f(n)$ 和 n^α 。
4. 判断主定理情形。
5. 写出 $T(n)$ 的 Θ 结果。

例题

例题 1

题目：求解：

$$T(n) = 3T(n/2) + n$$

解答:

这里:

$$b = 3$$

$$c = 2$$

$$\alpha = \log_2 3$$

$$f(n) = n$$

因为:

$$n = O(n^{\log_2 3 - \varepsilon})$$

可取 $\varepsilon = \log_2 3 - 1$ 。因此属于主定理第一种情形:

$$T(n) = \Theta(n^{\log_2 3})$$

例题 2

题目: 求解:

$$T(n) = 4T(n/2) + n^2$$

解答:

这里:

$$b = 4$$

$$c = 2$$

$$\alpha = \log_2 4 = 2$$

$$f(n) = n^2$$

因为:

$$f(n) = \Theta(n^\alpha)$$

属于主定理第二种情形:

$$T(n) = \Theta(n^2 \log n)$$

例题 3

题目：求解：

$$T(n) = 2T(n/2) + n^2$$

解答：

这里：

$$b = 2$$

$$c = 2$$

$$\alpha = \log_2 2 = 1$$

$$f(n) = n^2$$

$f(n)$ 多项式意义下大于 n^α 。检查正则条件：

$$2f(n/2) = 2(n/2)^2 = n^2/2 \leq (1/2)f(n)$$

取 $q = 1/2$ ，正则条件成立。属于主定理第三种情形：

$$T(n) = \Theta(n^2)$$

常见误区

第三种情形不能只比较大小，还要检查正则条件。

第二种情形要求同阶。若出现 $n^\alpha \log^k n$ ，需要按课堂给出的扩展主定理处理。

05 堆的定义、数组表示与操作

题目原文

了解堆的定义、如何用数组来表示堆和实现堆上的操作。

考查类型

概念题和算法设计题。

堆的定义

堆是一棵满足堆序性质的完全二叉树。

最大堆：

每个结点的值 $\geq$ 它的子结点的值

最小堆：

每个结点的值 $\leq$ 它的子结点的值

完全二叉树保证堆可以紧凑地存入数组，不需要显式保存指针。

数组表示

若数组从下标 1 开始：

$$parent(i) = \lfloor i/2 \rfloor$$

$$left(i) = 2i$$

$$right(i) = 2i + 1$$

若数组从下标 0 开始：

$$parent(i) = \lfloor (i - 1)/2 \rfloor$$

$$left(i) = 2i + 1$$

$$right(i) = 2i + 2$$

考试中要先说明使用哪一种下标体系，再写公式。

主要操作

向上调整

插入新元素时，把元素放在数组末尾。若它破坏堆序性质，就不断与父结点交换，直到堆序恢复。

时间复杂度：

$$O(\log n)$$

向下调整

删除堆顶时，用最后一个元素补到堆顶。若它破坏堆序性质，就不断与更合适的子结点交换，直到堆序恢复。

最大堆中应与较大的子结点交换。最小堆中应与较小的子结点交换。

时间复杂度：

$$O(\log n)$$

建堆

自底向上对所有非叶子结点做向下调整。

最后一个非叶子结点的位置为：

$$\lfloor n/2 \rfloor$$

这里采用 1 开始的数组下标。

总时间复杂度：

$$O(n)$$

典型伪代码

最大堆的向下调整：

MaxHeapify(A, i, heapSize):

$l = \text{left}(i)$

$r = \text{right}(i)$

$\text{largest} = i$

if $1 \leq \text{heapSize}$ and $A[1] > A[\text{largest}]$:

$\text{largest} = l$

```
if  $r \leq \text{heapSize}$  and  $A[r] > A[\text{largest}]$ :
```

$\text{largest} = r$

```
if  $\text{largest} \neq i$ :  
    swap  $A[i], A[\text{largest}]$   
    MaxHeapify( $A, \text{largest}, \text{heapSize}$ )
```

常见应用

优先队列可以用堆实现。

堆排序可以用最大堆实现：先建最大堆，再反复取出堆顶最大元素。

例题

例题 1

题目：判断数组是否表示一个最大堆。数组从下标 1 开始：

$A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

解答：

逐个检查非叶子结点：

$$16 \geq 14, 10$$

$$14 \geq 8, 7$$

$$10 \geq 9, 3$$

$$8 \geq 2, 4$$

$$7 \geq 1$$

每个父结点都不小于子结点，所以该数组表示一个最大堆。

例题 2

题目：向上面的最大堆插入 15，写出调整后的数组。

解答：

先把 15 放到数组末尾：

$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1, 15]$

15 的父结点是 7，交换：

$[16, 14, 10, 8, 15, 9, 3, 2, 4, 1, 7]$

15 的新父结点是 14, 继续交换:

[16, 15, 10, 8, 14, 9, 3, 2, 4, 1, 7]

此时 $15 \leq 16$, 调整结束。

例题 3

题目: 从原最大堆删除堆顶 16, 写出调整后的数组。

解答:

用末尾元素 1 补到堆顶, 再向下调整:

[1, 14, 10, 8, 7, 9, 3, 2, 4]

[14, 1, 10, 8, 7, 9, 3, 2, 4]

[14, 8, 10, 1, 7, 9, 3, 2, 4]

[14, 8, 10, 4, 7, 9, 3, 2, 1]

最终最大堆为:

[14, 8, 10, 4, 7, 9, 3, 2, 1]

常见误区

堆不是二叉搜索树。堆只保证父子之间的大小关系, 不保证左子树所有元素都小于右子树所有元素。

建堆不是 $O(n \log n)$ 的紧分析结果。逐个插入建堆是 $O(n \log n)$, 自底向上建堆是 $O(n)$ 。

06 快排平均复杂度递推分析

题目原文

复用快排中平均复杂度分析的方法来处理其它问题：

$$A(n) = (n - 1) + (2/n) \sum_{i=1}^{n-1} A(i)$$

考查类型

递推分析题。

递推来源

快速排序平均复杂度分析中，划分一次需要 $n - 1$ 次比较。若主元位置在所有位置上等概率出现，就会得到左右子问题规模的平均递推。

PDF 中给出的递推是：

$$A(n) = (n - 1) + (2/n) \sum_{i=1}^{n-1} A(i)$$

通常取：

$$A(1) = 0$$

解法：乘 n 再相减

原式两边乘以 n ：

$$nA(n) = n(n - 1) + 2 \sum_{i=1}^{n-1} A(i)$$

对 $n - 1$ 写同样的式子：

$$(n - 1)A(n - 1) = (n - 1)(n - 2) + 2 \sum_{i=1}^{n-2} A(i)$$

两式相减：

$$nA(n) - (n - 1)A(n - 1)$$

$$= 2(n - 1) + 2A(n - 1)$$

整理得：

$$nA(n) = (n+1)A(n-1) + 2(n-1)$$

两边除以 $n(n+1)$ ：

$$A(n)/(n+1) = A(n-1)/n + 2(n-1)/(n(n+1))$$

令：

$$B(n) = A(n)/(n+1)$$

则：

$$B(n) = B(n-1) + 2(n-1)/(n(n+1))$$

累加得到：

$$A(n) = 2(n+1)H_n - 4n$$

其中：

$$H_n = 1 + 1/2 + \dots + 1/n$$

由于：

$$H_n = \Theta(\log n)$$

所以：

$$A(n) = \Theta(n \log n)$$

答题模板

1. 写出平均递推。
2. 两边乘以 n ，消去分母。
3. 写出 $n-1$ 的同类递推。
4. 两式相减，去掉求和。
5. 变形为可累加形式。
6. 用调和级数 $H_n = \Theta(\log n)$ 得出结果。

例题

例题 1

题目：设：

$$A(1) = 0$$

$$A(n) = (n-1) + (2/n) \sum_{i=1}^{n-1} A(i)$$

计算 $A(2)$ 、 $A(3)$ 、 $A(4)$ 。

解答：

$$A(2) = 1 + (2/2)A(1) = 1$$

$$\begin{aligned} A(3) &= 2 + (2/3)(A(1) + A(2)) \\ &= 2 + (2/3)(0 + 1) \\ &= 8/3 \end{aligned}$$

$$\begin{aligned} A(4) &= 3 + (2/4)(A(1) + A(2) + A(3)) \\ &= 3 + (1/2)(0 + 1 + 8/3) \\ &= 29/6 \end{aligned}$$

例题 2

题目：用闭式公式验证 $A(4)$ 。

解答：

已知：

$$A(n) = 2(n+1)H_n - 4n$$

其中：

$$H_4 = 1 + 1/2 + 1/3 + 1/4 = 25/12$$

代入：

$$\begin{aligned} A(4) &= 2 \cdot 5 \cdot (25/12) - 16 \\ &= 250/12 - 192/12 \\ &= 29/6 \end{aligned}$$

与递推计算结果一致。

常见误区

不要直接把递推看成主定理形式。这里有求和项，不是 $T(n) = aT(n/b) + f(n)$ 。
不要漏掉 H_n 的渐进阶。结论的关键是调和级数带来的 $\log n$ 。

07 对抗策略分析

题目原文

能解释对抗策略的含义、能采用对抗策略来分析经典问题：同时求最大和最小的问题和求取第二大元素的问题。

考查类型

概念题和证明题。

对抗策略含义

对抗策略用于证明问题的下界。分析者假设存在一个对手，它在算法每次询问或比较时给出回答，并保持这些回答与至少一个合法输入一致。

如果对手能让任何算法都必须进行至少 $L(n)$ 次操作才能确定答案，则得到下界：

$$\Omega(L(n))$$

这种证明不是设计一个慢算法，而是证明所有算法都避不开一定数量的比较。

同时求最大值和最小值

朴素做法分别找最大和最小，需要：

$$(n - 1) + (n - 1) = 2n - 2$$

更优做法是成对比较。

步骤：

1. 每两个元素先相互比较一次。
2. 每对中较大的元素进入最大值比较范围。
3. 每对中较小的元素进入最小值比较范围。
4. 分别在两个范围内找最大和最小。

当 n 为偶数时，比较次数为：

$$n/2 + (n/2 - 1) + (n/2 - 1) = 3n/2 - 2$$

当 n 为奇数时，可以先留出一个元素作为初始最大值和最小值，再对剩余 $n - 1$ 个元素成对处理。比较次数为：

$$3(n - 1)/2$$

统一写法：

$$\lceil 3n/2 \rceil - 2$$

下界证明思路

每个元素在比较历史中需要获得足够信息：

一个元素只有输过一次，才可以排除它成为最大值。

一个元素只有赢过一次，才可以排除它成为最小值。

第一次参与比较时，一次比较能同时给两个元素分配角色：胜者不再是最小值，败者不再是最小值。之后要确定最大值和最小值，两个集合内还要分别完成淘汰。

由此得到同时求最大值和最小值的比较下界，与成对算法的次数匹配。

求第二大元素

先用锦标赛方式找最大值。每次比较的胜者继续向上，最终最大值会胜出。

找最大值需要：

$$n - 1$$

次比较。

第二大元素只会出现在直接输给最大值的元素中。若最大值直接比较并击败了 k 个元素，则还需要在这 k 个元素中找最大值，比较次数为：

$$k - 1$$

通过平衡锦标赛组织比较，可以使：

$$k = \lceil \log_2 n \rceil$$

总比较次数为：

$$\begin{aligned} n - 1 + \lceil \log_2 n \rceil - 1 \\ = n + \lceil \log_2 n \rceil - 2 \end{aligned}$$

第二大元素的下界思路

要确认最大值，至少需要 $n - 1$ 次比较，因为除了最大值外的每个元素都必须至少输一次。

要确认第二大元素，只需在直接输给最大值的元素中查找。对抗策略可使最大值必须直接击败至少 $\lceil \log_2 n \rceil$ 个元素，因此还需要至少 $\lceil \log_2 n \rceil - 1$ 次比较。

所以比较下界为：

$$n + \lceil \log_2 n \rceil - 2$$

例题

例题 1

题目： $n = 10$ 时，同时求最大值和最小值最少需要多少次比较？给出达到该次数的策略。

解答：

比较次数为：

$$\lceil 3n/2 \rceil - 2 = \lceil 15 \rceil - 2 = 13$$

策略：

1. 将 10 个元素两两配对，先比较 5 次。
2. 每对较大的 5 个元素中求最大值，需要 4 次比较。
3. 每对较小的 5 个元素中求最小值，需要 4 次比较。

总次数：

$$5 + 4 + 4 = 13$$

例题 2

题目：用锦标赛方法在下面数组中求第二大元素，并写出比较次数。

[9, 4, 7, 12, 3, 10, 6, 8]

解答：

第一轮：

$$9 > 4$$

$$12 > 7$$

$$10 > 3$$

$$8 > 6$$

第二轮：

$$12 > 9$$

$$10 > 8$$

第三轮：

$$12 > 10$$

最大值是 12。直接输给 12 的元素是：

7, 9, 10

在这三个元素中再找最大值，需要 2 次比较，得到第二大元素：

10

总比较次数：

$$7 + 2 = 9$$

公式验证：

$$n + \lceil \log_2 n \rceil - 2 = 8 + 3 - 2 = 9$$

常见误区

第二大元素不是在所有非最大元素中重新找最大。只需要检查直接输给最大值的元素。
对抗策略证明的是任何算法都需要的最低比较次数，不是某个具体算法的运行过程。

08 线性时间选择算法

题目原文

线性时间选择算法的实现和应用

考查类型

算法设计题和复杂度分析题。

问题定义

选择问题是：给定 n 个元素和整数 k ，找出第 k 小的元素。

特殊情况：

$k = 1$	找最小值
$k = n$	找最大值
$k = (n+1)/2$	找中位数

随机选择算法思想

随机选择与快速排序的划分类似：

1. 选一个划分基准。
2. 按基准把数组分成小于、等于、大于三部分。
3. 根据 k 所在范围，只递归进入一边。

平均时间复杂度：

$O(n)$

最坏时间复杂度：

$O(n^2)$

确定性线性时间选择算法

也称中位数的中位数方法。

步骤：

1. 将 n 个元素每 5 个分成一组。
2. 对每组内部排序，取每组中位数。

3. 递归地从这些中位数中选出中位数，作为划分基准。
4. 用该基准对原数组划分。
5. 根据 k 的位置，只递归进入一边。

为什么是线性时间

每组 5 个元素。取所有组的中位数后，再取这些中位数的中位数作为划分基准。

至少一半的组中位数不小于该基准。每个这样的组中，至少有 3 个元素不小于该组中位数，因此也不小于该基准。取整项只影响常数，不影响线性时间结论。

由此可保证每次递归至少排除常数比例的元素。

递推可写为：

$$T(n) \leq T(n/5) + T(7n/10) + O(n)$$

解得：

$$T(n) = O(n)$$

应用

找中位数：使用 $\text{Select}(A, \lceil n/2 \rceil)$ 。

找第 k 小：

$\text{Select}(A, k)$

找前 k 小元素：

先用选择算法找到第 k 小的元素，再按该元素划分数组。

找分位点：

递归使用选择算法找多个秩位置的元素。

伪代码框架

```
Select(A, k):
    if |A| is small:
        sort A
        return A[k]
```

split A into groups of 5

$M = \text{medians of all groups}$

$p = \text{Select}(M, \lceil |M|/2 \rceil)$


```

partition A into L, E, G by p

if  $k \leq |L|$ :
    return Select(L, k)
else if  $k \leq |L| + |E|$ :
    return p
else:
    return Select(G,  $k - |L| - |E|$ )

```

例题

例题 1

题目：在数组中找第 5 小元素：

$A = [9, 1, 7, 3, 8, 2, 6, 5, 4]$

解答：

按升序排列用于核对结果：

$[1, 2, 3, 4, 5, 6, 7, 8, 9]$

第 5 小元素为：

5

选择算法不需要完整排序。一次以 5 为基准划分：

$L = [1, 3, 2, 4]$

$E = [5]$

$G = [9, 7, 8, 6]$

因为：

$$|L| = 4$$

$$|L| + |E| = 5$$

第 5 小元素正好落在 E 中，答案是 5。

例题 2

题目：对下面数组使用中位数的中位数方法选取划分基准，并求第 8 小元素。

$[12, 5, 7, 1, 9, 3, 14, 6, 10, 2, 8, 15, 4, 13, 11]$

解答：

每 5 个一组：

[12, 5, 7, 1, 9]	中位数 7
[3, 14, 6, 10, 2]	中位数 6
[8, 15, 4, 13, 11]	中位数 11

中位数组为：

[7, 6, 11]

其中位数是 7，取 7 为划分基准。

划分后：

L = [5, 1, 3, 6, 2, 4]
E = [7]
G = [12, 9, 14, 10, 8, 15, 13, 11]

因为：

$$|L| = 6$$

$$|L| + |E| = 7$$

第 8 小元素在 G 中，并且是 G 的第 1 小元素。G 的最小值为：

8

因此原数组第 8 小元素是 8。

常见误区

线性时间选择算法不需要完整排序。完整排序是 $O(n \log n)$ ，选择算法只递归处理一边。

每组大小取 5 是为了保证递推能收敛到线性时间。组大小不是随意选择的。

09 平摊分析

题目原文

能解释平摊分析的含义，能采用平摊分析解决经典问题：MultiPop Stack、DeleteLargerHalf 和栈模拟队列问题

考查类型

概念题和分析题。

平摊分析含义

平摊分析研究一串操作的总成本，再把总成本分摊到每次操作上。

它不依赖输入概率分布，因此不同于平均复杂度。

若任意 m 次操作的总成本为 $O(m)$ ，则每次操作的平摊成本为：

$$O(1)$$

三种常见方法

聚合分析：

直接证明一串操作总成本的上界。

记账法：

给某些操作多收费用，把多收的费用存起来支付后续昂贵操作。

势能法：

定义势能函数 $\Phi(D)$ ，用结构状态中保存的势能解释未来成本。

平摊代价：

$$amortizedcost = actualcost + \Phi(after) - \Phi(before)$$

MultiPop Stack

操作：

Push(x)

Pop()

MultiPop(k)

MultiPop(k) 连续弹出最多 k 个元素，实际成本与弹出数量有关。

聚合分析：

每个元素最多被压栈一次，也最多被弹出一次。无论由 Pop 弹出，还是由 MultiPop 弹出，同一个元素只会支付一次弹出成本。

因此，在 m 次操作中，总压栈次数不超过 m ，总弹出次数也不超过 m 。

总成本：

$$O(m)$$

每次操作平摊：

$$O(1)$$

DeleteLargerHalf

PDF 未给出完整操作定义。以下小节只分析本文件采用的练习模型：当前规模为 m 时，删除 $\lfloor m/2 \rfloor$ 个较大元素；一次操作的成本按本次扫描元素数或删除元素数计。课堂定义不同，则需要按课堂定义重写该小节。

分析要点：

1. 一次 DeleteLargerHalf 看起来很贵。
2. 但它会使当前元素数量从 m 变为 $\lceil m/2 \rceil$ 。
3. 对一段连续删除操作，总规模形成几何级数。

若开始时有 n 个元素，连续执行 DeleteLargerHalf 的总删除规模满足：

$$n/2 + n/4 + n/8 + \dots \leq n$$

因此总成本为：

$$O(n)$$

若操作序列中还包含插入操作，可以把删除成本分摊给被删除元素；每个元素在被插入后最多被删除一次，所以整段操作仍可用聚合分析或记账法处理。

两个栈模拟队列

使用两个栈：

S_in 负责入队
S_out 负责出队

入队：

push 到 S_in

出队：

```

if S_out 非空:
    pop S_out
else:
    把 S_in 中所有元素依次弹出并压入 S_out
    pop S_out

```

单次出队在转移时成本为 $O(n)$ ，但每个元素只会：

1. 压入 S_{in} 一次。
2. 从 S_{in} 弹出一次。
3. 压入 S_{out} 一次。
4. 从 S_{out} 弹出一次。

所以每个元素总共承担常数栈操作。

任意 m 次队列操作总成本：

$O(m)$

每次操作平摊：

$O(1)$

例题

例题 1

题目：分析下面栈操作序列的总成本：

Push(1), Push(2), Push(3), MultiPop(2), Push(4), Pop()

解答：

逐项成本：

Push(1)	1
Push(2)	1
Push(3)	1
MultiPop(2)	2
Push(4)	1
Pop()	1

总成本：

$$1 + 1 + 1 + 2 + 1 + 1 = 7$$

共有 6 次操作，单次实际成本并不都相同，但总成本与操作数同阶。对任意操作序列，每个元素最多压入一次、弹出一次，因此平摊成本为 $O(1)$ 。

例题 2

题目：一个集合开始有 16 个元素。每次 DeleteLargerHalf 扫描当前全部元素，并删除较大的一半。连续执行直到只剩 1 个元素，求总扫描成本。

解答：

每次操作前的规模为：

16, 8, 4, 2

总扫描成本为：

$$16 + 8 + 4 + 2 = 30$$

因为：

$$16 + 8 + 4 + 2 < 32 = 2 \cdot 16$$

所以连续删除操作的总成本为 $O(n)$ 。

例题 3

题目：用两个栈模拟队列，分析下面操作序列的栈操作次数：

Enqueue(1), Enqueue(2), Enqueue(3), Dequeue(), Dequeue(), Enqueue(4),
Dequeue()

解答：

前三次入队各做一次压栈，共 3 次栈操作。

第一次出队时 S_{out} 为空，需要把 S_{in} 中 3 个元素转移到 S_{out} ，再弹出队首：

3 次从 S_{in} 弹出
3 次压入 S_{out}
1 次从 S_{out} 弹出

共 7 次栈操作。

第二次出队直接从 S_{out} 弹出，成本 1。

Enqueue(4) 成本 1。

第三次出队直接从 S_{out} 弹出，成本 1。

总栈操作次数：

$$3 + 7 + 1 + 1 + 1 = 13$$

每个入队元素只在两个栈之间移动一次，因此任意长序列的平摊成本为 $O(1)$ 。

常见误区

平摊复杂度不是平均复杂度。平摊分析不需要概率。

某一次操作实际成本可以很高。平摊结论描述的是整段操作序列的均摊成本。

10 DFS 与 BFS 图搜索

题目原文

简述深度优先和广度优先算法的思想，采用该算法思想能解决经典的图上搜索问题：topo 序、任务调度、关键路径和强联通区域问题

考查类型

概念题和算法设计题。

DFS 思想

深度优先搜索从一个起点出发，沿着一条路径尽量向深处走；当没有未访问邻接点时，再回退到上一个结点继续搜索。

实现方式：

递归
显式栈

典型记录：

visited[v]
discover time
finish time
parent[v]

时间复杂度：

$O(V + E)$

BFS 思想

广度优先搜索从起点出发，先访问距离为 1 的点，再访问距离为 2 的点，按层推进。

实现方式：

队列

典型记录：

visited[v]
distance[v]
parent[v]

时间复杂度：

$O(V + E)$

BFS 可求无权图中从起点到各点的最短路径长度。

topo 序

拓扑序用于有向无环图。

DFS 做法：

1. 对图做 DFS。
2. 按完成时间从大到小排列顶点。
3. 得到拓扑序。

Kahn 算法做法：

1. 统计所有顶点入度。
2. 将入度为 0 的顶点入队。
3. 每取出一个顶点，就删除它的出边并更新邻点入度。
4. 新的入度为 0 的顶点入队。

若最终输出顶点数少于 v ，说明图中存在环。

任务调度

任务依赖关系可建模为有向图：

$u \rightarrow v$ 表示任务 u 必须先于任务 v 完成

若图是 DAG，则拓扑序给出合法任务执行顺序。

若图有环，则依赖关系冲突，无法完成全部任务。

关键路径

关键路径通常在有向无环图中讨论。边或点带有持续时间。

基本步骤：

1. 求拓扑序。
2. 按拓扑序计算每个点的最早开始或最早完成时间。
3. 反向按逆拓扑序计算最晚开始或最晚完成时间。
4. 松弛时间为 0 的活动位于关键路径上。

关键路径长度决定项目最短完成时间。

强连通区域问题

有向图中，若两个顶点互相可达，则它们属于同一个强连通分量。

Kosaraju 算法：

1. 对原图做 DFS，记录完成时间。
2. 将所有边反向。
3. 按原图完成时间从大到小，在反图上 DFS。
4. 每次 DFS 访问到的一组顶点构成一个强连通分量。

Tarjan 算法也使用 DFS，通过 dfn 和 low 维护强连通分量。

例题

例题 1

题目：给定有向图：

$$V = A, B, C, D, E, F$$

$$E = A \rightarrow C, B \rightarrow C, B \rightarrow D, C \rightarrow E, D \rightarrow F, E \rightarrow F$$

写出一个拓扑序。

解答：

一个合法拓扑序为：

A, B, C, D, E, F

检查每条边：

A 在 C 前
B 在 C 前
B 在 D 前
C 在 E 前
D 在 F 前
E 在 F 前

所有依赖均满足，因此该序列是拓扑序。

例题 2

题目：在无权图中从 s 出发做 BFS。边集合为：

s-a, s-b, a-c, b-c, c-d, b-e

求从 s 到各点的最短距离。

解答：

BFS 按层推进：

第 0 层： s

第 1 层： a, b

第 2 层： c, e

第 3 层： d

距离为：

$\text{dist}[s] = 0$

$\text{dist}[a] = 1$

$\text{dist}[b] = 1$

$\text{dist}[c] = 2$

$\text{dist}[e] = 2$

$\text{dist}[d] = 3$

例题 3

题目：求下面有向图的强连通分量：

$1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 5, 5 \rightarrow 4$

解答：

顶点 1, 2, 3 互相可达，构成一个强连通分量：

$\{1, 2, 3\}$

顶点 4, 5 互相可达，构成一个强连通分量：

$\{4, 5\}$

边 $3 \rightarrow 4$ 只提供从第一个分量到第二个分量的方向，不能让两个分量合并。

常见误区

BFS 适合无权图最短路，不适合直接处理带权最短路。

拓扑序只对 DAG 存在。若图有有向环，则不存在拓扑序。

强连通分量是有向图概念，不是无向连通分量。

11 贪心法

题目原文

贪心法的思想和特点，用贪心法解决经典问题并能给出证明思路：换硬币、单源最短路径。

考查类型

概念题、算法设计题和证明题。

贪心法思想

贪心法在每一步都做当前看来最优的选择，并希望这些局部选择组合成全局最优解。
适用贪心法通常需要两个性质：

贪心选择性质
最优子结构

贪心选择性质表示存在一个全局最优解包含当前的局部最优选择。

最优子结构表示做出一次选择后，剩余问题的最优解可以和当前选择组合成原问题最优解。

证明思路

常用证明方法有两类。

交换论证：

从任意最优解出发，把它逐步替换成包含贪心选择的最优解，同时不降低解的质量。

领先性证明：

证明贪心算法在每一步都不差于任何最优解的对应前缀。

换硬币问题

目标是在给定币值系统下，用尽量少的硬币凑出金额 N 。

贪心策略：

每次选择不超过剩余金额的最大面值硬币

重要点：

不是所有币值系统都能用贪心得最优解。答题时必须结合题目给出的币值系统证明贪心选择性质。

证明结构：

1. 说明贪心第一步选取最大可用面值。
2. 证明存在一个最优解也包含这枚硬币。
3. 删除这枚硬币后，剩余金额构成同类子问题。
4. 对子问题重复使用同样论证。

若题目给出标准币值系统，需要用该系统的面值关系证明“大面值替换多个小面值不会增加硬币数”。

单源最短路径

单源最短路径的典型贪心算法是 Dijkstra 算法。它要求边权非负。

算法思想：

1. 维护源点到各点的当前最短估计 $\text{dist}[v]$ 。
2. 每次从未确定顶点中取 dist 最小的顶点 u 。
3. 将 u 标记为已确定。
4. 用 u 的出边松弛相邻顶点。

伪代码：

```
Dijkstra(G, s):
    for each vertex v:
         $\text{dist}[v] = \infty$ 
     $\text{dist}[s] = 0$ 

     $Q = \text{allvertices}$ 

    while Q is not empty:
         $u = \text{vertex in } Q \text{ with minimum } \text{dist}[u]$ 
        remove u from Q
        for each edge (u, v):
            if  $\text{dist}[v] > \text{dist}[u] + w(u, v)$ :
                 $\text{dist}[v] = \text{dist}[u] + w(u, v)$ 
```

Dijkstra 正确性证明

关键命题：

每次取出的 dist 最小未确定顶点 u ，其 $\text{dist}[u]$ 已经等于源点到 u 的真实最短距离。

证明思路：

若存在一条更短路径从源点到 u ，考虑这条路径上第一个未确定顶点 y ，以及它前一个已确定顶点 x 。由于 x 已经确定，边 (x, y) 已经被松弛，因此 $\text{dist}[y]$ 不会大于该路

径到 y 的长度。边权非负，所以到 y 的长度不大于到 u 的长度。这会与 u 是未确定顶点中 dist 最小者矛盾。

因此贪心选择正确。

例题

例题 1

题目：币值为：

1, 2, 4, 8, 16

用贪心法凑出金额 23，并说明最优性。

解答：

贪心选择：

$$23 = 16 + 4 + 2 + 1$$

共用 4 枚硬币。

这些币值都是 2 的幂。金额 23 的二进制表示为：

$$23 = 16 + 4 + 2 + 1$$

每个二进制位对应一枚硬币。使用更小面值替代其中任意一枚硬币，都不会减少硬币数。因此该贪心解是最优解。

例题 2

题目：对下面非负权图从源点 s 运行 Dijkstra 算法：

$$s \rightarrow a : 1$$

$$s \rightarrow b : 4$$

$$a \rightarrow b : 2$$

$$a \rightarrow c : 5$$

$$b \rightarrow c : 1$$

$$b \rightarrow d : 7$$

$$c \rightarrow d : 3$$

求从 s 到各点的最短距离。

解答：

初始化：

```
dist[s] = 0
dist[a] = ∞
dist[b] = ∞
dist[c] = ∞
dist[d] = ∞
```

从 s 松弛:

```
dist[a] = 1
dist[b] = 4
```

取出 a, 松弛:

```
dist[b] = min(4, 1 + 2) = 3
dist[c] = 1 + 5 = 6
```

取出 b, 松弛:

```
dist[c] = min(6, 3 + 1) = 4
dist[d] = 3 + 7 = 10
```

取出 c, 松弛:

```
dist[d] = min(10, 4 + 3) = 7
```

最终结果:

```
dist[s] = 0
dist[a] = 1
dist[b] = 3
dist[c] = 4
dist[d] = 7
```

常见误区

贪心算法必须证明正确性，不能只写“每一步取最优”。

Dijkstra 算法不能直接用于含负权边的图。

换硬币问题不能默认所有币值系统都适合贪心。

12 动态规划

题目原文

动态规划的思想 and 特点，针对经典问题能写出子问题的递归式，描述算法思想和伪代码：最长公共子序列相关问题、抽牌拿取最大值问题、回文判断/分割相关问题。

考查类型

概念题和算法设计题。

动态规划思想

动态规划用于具有以下性质的问题：

最优子结构
重叠子问题

做法是先定义状态，再写状态转移，最后确定计算顺序。

答题四步：

1. 定义 dp 状态含义。
2. 写初始条件。
3. 写状态转移方程。
4. 说明遍历顺序和最终答案。

最长公共子序列

给定字符串 $X[1..m]$ 和 $Y[1..n]$ ，令：

$dp[i][j]$ = $X[1..i]$ 和 $Y[1..j]$ 的最长公共子序列长度

初始条件：

$dp[0][j] = 0$
 $dp[i][0] = 0$

状态转移：

```
if  $X[i] == Y[j]$ :  
     $dp[i][j] = dp[i-1][j-1] + 1$   
else:  
     $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ 
```


答案:

$dp[m][n]$

时间复杂度:

$O(mn)$

空间复杂度:

$O(mn)$

若只要求长度, 可以滚动数组优化为 $O(n)$ 空间。

抽牌拿取最大值问题

PDF 未给出完整题面。以下小节只分析本文件采用的练习模型: 一排牌, 每次只能从左端或右端取一张, 两人都采用最优策略, 求先手能取得的最大收益或最大分差。课堂题面不同, 则需要按课堂题面重写该小节。

定义:

$dp[i][j]$ = 当前行动者面对区间 $cards[i..j]$ 时, 相对对手最多领先多少分

状态转移:

```
dp[i][j] = max(  
    cards[i] - dp[i+1][j],  
    cards[j] - dp[i][j-1]  
)
```

含义:

当前行动者若取左端 $cards[i]$, 接下来对手面对 $[i+1..j]$, 对手相对当前行动者的领先为 $dp[i+1][j]$, 所以当前行动者净领先为 $cards[i] - dp[i+1][j]$ 。

初始条件:

$dp[i][i] = cards[i]$

计算顺序:

区间长度从 1 到 n

答案:

$dp[0][n-1]$

若需要先手最大总分, 可结合总分 S 得到:

$first = (S + dp[0][n-1]) / 2$

回文判断

给定字符串 $s[0..n-1]$, 令:

$pal[i][j] = s[i..j]$ 是否为回文串

初始和转移:

```
pal[i][i] = true
pal[i][i+1] = (s[i] == s[i+1])

if s[i] == s[j] and pal[i+1][j-1]:
    pal[i][j] = true
else:
    pal[i][j] = false
```

计算顺序:

按区间长度从短到长

回文分割

目标: 将字符串切成若干回文子串, 求最少切割次数。

先用上面的 $pal[i][j]$ 预处理任意区间是否回文。

令:

$cut[i] = s[0..i]$ 的最少切割次数

转移:

```
if pal[0][i]:
    cut[i] = 0
else:
    cut[i] = min(cut[j] + 1) for  $0 \leq j < i$  and  $pal[j+1][i]$ 
```

时间复杂度:

$$O(n^2)$$

例题

例题 1

题目: 求字符串 $X = ABCBDAB$ 和 $Y = BDCABA$ 的最长公共子序列长度。

解答:

令:

$dp[i][j]$ = $X[1..i]$ 和 $Y[1..j]$ 的最长公共子序列长度

按转移方程填表后得到：

$$dp[7][6] = 4$$

因此最长公共子序列长度为 4。一个长度为 4 的公共子序列为：

BCBA

例题 2

题目：一排牌为：

[3, 9, 1, 2]

每次只能从左端或右端取牌，两人都采用最优策略。求先手最终得分。

解答：

沿用上文定义：

$dp[i][j]$ = 当前行动者面对 $cards[i..j]$ 时，相对对手最多领先多少分

单元素区间：

$$dp[0][0] = 3$$

$$dp[1][1] = 9$$

$$dp[2][2] = 1$$

$$dp[3][3] = 2$$

长度为 2：

$$dp[0][1] = \max(3 - 9, 9 - 3) = 6$$

$$dp[1][2] = \max(9 - 1, 1 - 9) = 8$$

$$dp[2][3] = \max(1 - 2, 2 - 1) = 1$$

长度为 3：

$$dp[0][2] = \max(3 - 8, 1 - 6) = -5$$

$$dp[1][3] = \max(9 - 1, 2 - 8) = 8$$

长度为 4：

$$dp[0][3] = \max(3 - 8, 2 - (-5)) = 7$$

总分为：

$$3 + 9 + 1 + 2 = 15$$

先手得分为：

$$(15 + 7)/2 = 11$$

例题 3

题目：对字符串 aab 求最少回文分割次数。

解答：

回文子串包括：

a
a
b
aa

最优分割为：

aa | b

切割次数为：

1

用 cut 表示：

cut[0] = 0 a
cut[1] = 0 aa
cut[2] = cut[1] + 1 = 1

所以答案为 1。

常见误区

动态规划不是只写递归式。完整答案还要说明状态含义、边界条件、计算顺序和最终答案。

区间动态规划要按区间长度递增计算，避免使用尚未计算出的状态。

最长公共子序列不是最长公共子串。子序列允许不连续，子串必须连续。